



C o m m u n i t y E x p e r i e n c e D i s t i l l e d

Mastering matplotlib

A practical guide that takes you beyond the basics of matplotlib
and gives solutions to plot complex data

Duncan M. McGregor

[PACKT] open source*
PUBLISHING community experience distilled

Mastering matplotlib

A practical guide that takes you beyond the basics of matplotlib and gives solutions to plot complex data

Duncan M. McGregor



BIRMINGHAM - MUMBAI

Mastering matplotlib

Copyright © 2015 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing, and its dealers and distributors will be held liable for any damages caused or alleged to be caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

First published: June 2015

Production reference: 1250615

Published by Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham B3 2PB, UK.

ISBN 978-1-78398-754-2

www.packtpub.com

Credits

Author

Duncan M. McGregor

Project Coordinator

Shweta H. Birwatkar

Reviewers

Francesco Benincasa

Wen-Wei Liao

Nicolas P. Rougier

Dr. Allen Chi-Shing Yu

Proofreader

Safis Editing

Indexer

Hemangini Bari

Acquisition Editor

Meeta Rajani

Graphics

Sheetal Aute

Content Development Editor

Sumeet Sawant

Production Coordinator

Komal Ramchandani

Technical Editor

Gaurav Suri

Cover Work

Komal Ramchandani

Copy Editors

Ulka Manjrekar

Vedangi Narvekar

About the Author

Duncan M. McGregor, having programmed with GOTOs in the 1980s, has made up for that through community service by making open source contributions for more than 20 years. He has spent a major part of the past 10 years dealing with distributed and scientific computing (in languages ranging from Python, Common Lisp, and Julia to Clojure and Lisp Flavored Erlang). In the 1990s, after serving as a linguist in the US Army, he spent considerable time working on projects related to MATLAB and Mathematica, which was a part of his physics and maths studies at the university. Since the mid 2000s, matplotlib and NumPy have figured prominently in many of the interesting problems that he has solved for his customers. With the most recent addition of the IPython Notebook, matplotlib and the suite of the Python scientific computing libraries remain some of his most important professional tools.

About the Reviewers

Francesco Benincasa, master of science in software engineering, is a software designer and developer. He is a GNU/Linux and Python expert and has vast experience in many other languages and applications. He has been using Python as the primary language for more than 10 years, together with JavaScript and frameworks such as Plone, Django, and JQuery.

He is interested in advanced web and network developing as well as scientific data manipulation, analysis, and visualization. Over the last few years, he has been using graphical Python libraries such as matplotlib/Basemap, scientific libraries such as NumPy/SciPy, Pandas, and PyTables, and scientific applications such as GrADS, NCO, and CDO.

He is currently working at the Earth Sciences Department of the Barcelona Supercomputing Center (www.bsc.es) as a research support engineer. He is involved in projects such as the World Meteorological Organization Sand and Dust Storms Warning Advisory and Assessment System (<http://sds-was.aemet.es/>) and the Barcelona Dust Forecast Center (<http://dust.aemet.es/>).

He has already worked for Packt Publishing in the past as a reviewer for *matplotlib Plotting Cookbook*.

I would like to thank my wonderful future wife, Francesca, for her constant support and love.

Wen-Wei Liao received his MSc in systems neuroscience from National Tsing Hua University, Taiwan. He is interested in the development of computational strategies to interpret the genomic and epigenomic data that is produced from high-throughput sequencing. He works as a computational science developer at the Cold Spring Harbor Laboratory. More information regarding him can be found at <http://wwliao.name/>.

Nicolas P. Rougier is a researcher at INRIA (France), which is the French national institute for research in computer science and control. His research lies at the frontier between integrative and computational neuroscience, where he tries to understand higher brain functions using computational models. He also has experience in scientific visualization and has produced several tutorials (matplotlib tutorials, NumPy tutorials, and 100 NumPy exercises) as well as the popular *Ten Simple Rules for Better Figures* article.

Dr. Allen Chi-Shing Yu is a postdoctoral fellow who is currently working in the field of cancer genetics. He obtained his BSc degree in molecular biotechnology at the Chinese University of Hong Kong (CUHK) in 2009 and a PhD degree in biochemistry at the same university in 2013. In 2010, Allen led the first team in CUHK to join MIT's prestigious International Genetically Engineered Machine (iGEM) competition. His team, a 2010 iGEM gold medalist, worked on using bacteria as an obfuscated massive data storage device. The project was widely covered by the media, including AFP, Engadget, PopSci, and Time, to name a few.

His thesis research primarily involves the characterization of novel bacterial strains that can use toxic fluoro-tryptophans, but not the canonical tryptophan, for propagation. The findings demonstrated that the genetic code is not an immutable construct despite billions of years of invariance. Soon after these microbial studies, he identified and characterized a novel marker that causes Spinocerebellar Ataxia (SCA), which is a group of diverse neurodegenerative disorders. This research about the novel SCA marker was recently published in the Journal of Medical Genetics. Recently, through the development of a tool that was used to detect viral integration events in human cancer samples (ViralFusionSeq), he entered the field of cancer genetics. As a postdoctoral fellow in Professor Nathalie Wong's lab, he is now taking part in the analysis of hepatocellular carcinoma using the data from the high-throughput sequencing of genomes and transcriptomes.

Special thanks to Dorothy for her love and support!

www.PacktPub.com

Support files, eBooks, discount offers, and more

For support files and downloads related to your book, please visit www.PacktPub.com.

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at www.PacktPub.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at service@packtpub.com for more details.

At www.PacktPub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters and receive exclusive discounts and offers on Packt books and eBooks.



<https://www2.packtpub.com/books/subscription/packtlib>

Do you need instant solutions to your IT questions? PacktLib is Packt's online digital book library. Here, you can search, access, and read Packt's entire library of books.

Why subscribe?

- Fully searchable across every book published by Packt
- Copy and paste, print, and bookmark content
- On demand and accessible via a web browser

Free access for Packt account holders

If you have an account with Packt at www.PacktPub.com, you can use this to access PacktLib today and view 9 entirely free books. Simply use your login credentials for immediate access.

Table of Contents

Preface	vii
Chapter 1: Getting Up to Speed	1
A brief historical overview of matplotlib	2
What's new in matplotlib 1.4	2
The intermediate matplotlib user	2
Prerequisites for this book	3
Python 3	4
Coding style	5
Installing matplotlib	6
Using IPython Notebooks with matplotlib	6
Advanced plots – a preview	7
Setting up the interactive backend	8
Joint plots with Seaborn	8
Scatter plot matrix graphs with Pandas	10
Summary	11
Chapter 2: The matplotlib Architecture	13
The original design goals	14
The current matplotlib architecture	14
The backend layer	16
FigureCanvasBase	17
RendererBase	18
Event	18
Visualizing the backend layer	19
The artist layer	20
Primitives	21
Containers	22
Collections	22
A view of the artist layer	23

The scripting layer	24
The supporting components of the matplotlib stack	26
matplotlib modules	28
Exploring the filesystem	28
Exploring imports visually	32
ModuleFinder	32
ModGrapher	33
The execution flow	37
An overview of the script	37
An interactive session	39
The matplotlib architecture as it relates to this book	43
Summary	44
Chapter 3: matplotlib APIs and Integrations	45
The procedural pylab API	46
The pyplot scripting API	52
The matplotlib object-oriented API	57
Equations	58
Helper classes	59
The Plotter class	61
Running the jobs	63
matplotlib in other frameworks	66
An important note on IPython	67
Summary	68
Chapter 4: Event Handling and Interactive Plots	69
Event loops in matplotlib	70
Event-based systems	71
The event loop	71
GUI toolkit main loops	74
IPython Notebook event loops	75
matplotlib event loops	76
Event handling	77
Mouse events	78
Keyboard events	81
Axes and figure events	86
Object picking	86
Compound event handling	87
The navigation toolbar	87
Specialized events	89
Interactive panning and zooming	89
Summary	94

Chapter 5: High-level Plotting and Data Analysis	95
High-level plotting	96
Historical background	97
matplotlib	99
NetworkX	100
Pandas	103
The grammar of graphics	105
Bokeh	106
The yhat ggplot	106
New styles in matplotlib	108
Seaborn	109
Data analysis	111
Pandas, SciPy, and Seaborn	112
Examining and shaping a dataset	112
Analysis of temperature	115
Analysis of precipitation	136
Summary	140
Chapter 6: Customization and Configuration	141
Customization	142
Creating a custom style	142
Subplots	145
Revisiting Pandas	147
Individual plots	148
Bringing everything together	151
Further explorations in customization	157
Configuration	157
The run control for matplotlib	158
File and directory locations	158
Using the matplotlibrc file	158
Updating the settings dynamically	160
Options in IPython	160
Summary	161
Chapter 7: Deploying matplotlib in Cloud Environments	163
Making a use case for matplotlib in the Cloud	164
The data source	164
Defining a workflow	165
Choosing technologies	171
Configuration management	172
Types of deployment	173
An example – AWS and Docker	173
Getting set up locally	173
Requirements	174

Dockerfiles and the Docker images	174
Extending a Docker image	176
Building a new image	178
Preparing for deployment	179
Getting the setup on AWS	179
Pushing the source data to S3	180
Creating a host server on EC2	181
Using Docker on EC2	183
Reading and writing with S3	183
Running the task	185
Environment variables and Docker	185
Changes to the Python module	186
Execution	188
Summary	190
Chapter 8: matplotlib and Big Data	191
Big data	192
Working with large data sources	193
An example problem	194
Big data on the filesystem	196
NumPy's memmap function	196
HDF5 and PyTables	202
Distributed data	209
MapReduce	210
Open source options	215
An example – working with data on EMR	216
Visualizing large data	225
Finding the limits of matplotlib	226
Agg rendering with matplotlibrc	229
Decimation	231
Additional techniques	232
Other visualization tools	233
Summary	234
Chapter 9: Clustering for matplotlib	235
Clustering and parallel programming	236
The custom ZeroMQ cluster	237
Estimating the value of π	238
Creating the ZeroMQ components	241
Working with the results	249
Clustering with IPython	252
Getting started	253
The direct view	254
The load-balanced view	256

The parallel magic functions	258
An example – estimating the value of π	258
More clustering	260
Summary	260
Index	263

Preface

In just over a decade, matplotlib has grown to offer the Python scientific computing community a world-class plotting and visualization library. When combined with related projects, such as Jupyter, NumPy, SciPy, and SymPy, matplotlib competes head-to-head with commercial software, which is far more established in the industry. Furthermore, the growth experienced by this open source software project is reflected again and again by individuals around the world, who make their way through the thorny wilds that face the newcomer and who develop into strong intermediate users with the potential to be very productive.

In essence, *Mastering matplotlib* is a very practical book. Yet every chapter was written considering this learning process, as well as a larger view of the same. It is not just the raw knowledge that defines how far developers progress in their goal. It is also the ability of motivated individuals to apply meta-levels of analysis to the problem and the obstacles that must be surmounted. Implicit in the examples that are provided in each chapter are multiple levels of analysis, which are integral to the mastery of the subject matter. These levels of analysis involve the processes of defining the problem, anticipating potential solutions, evaluating approaches without losing focus, and enriching your experience with a wider range of useful projects.

Finding resources that facilitate developers in their journey towards advanced knowledge and beyond can be difficult. This is not due to the lack of materials. Rather, it is because of the complex interaction of learning styles, continually improving codebases with strong legacies, and the very flexible nature of the Python programming language itself. The matplotlib developers who aspire to attain an advanced level, must tackle all of this and more. This book aims to be a guide for those in search of such mastery.

What this book covers

Chapter 1, Getting Up to Speed, covers some history and background of matplotlib, goes over some of the latest features of the library, provides a refresher on Python 3 and IPython Notebooks, and whets the reader's appetite with some advanced plotting examples.

Chapter 2, The matplotlib Architecture, reviews the original design goals of matplotlib and then proceeds to discuss its current architecture in detail, providing visualizations of the conceptual structure and relationships between the Python modules.

Chapter 3, matplotlib APIs and Integrations, walks the reader through the matplotlib APIs, adapting a single example accordingly, examines how third-party libraries are integrated with matplotlib, and gives migration advice to the advanced users of the deprecated pylab API.

Chapter 4, Event Handling and Interactive Plots, provides a review of the event-based systems, covers event loops in matplotlib and IPython, goes over a selection of matplotlib events, and shows how to take advantage of these to create interactive plots.

Chapter 5, High-level Plotting and Data Analysis, combines the interrelated topics, providing a historical background of plotting, a discussion on the grammar of graphics, and an overview of high-level plotting libraries. This is then put to use in a detailed analysis of weather-related data that spans 120 years.

Chapter 6, Customization and Configuration, covers the custom styles in matplotlib and the use of grid specs to create a dashboard effect with the combined plots. The lesser-known configuration options are also discussed with an eye to optimization.

Chapter 7, Deploying matplotlib in Cloud Environments, explores a use case for matplotlib in a remote deployment, which is followed by a detailed programmatic batch-job example using Docker and Amazon AWS.

Chapter 8, matplotlib and Big Data, provides detailed examples of working with large local data sets, as well as distributed ones, covering options such as `numpy.memmap`, HDF5, and Hadoop. Plots with millions of points will also be demonstrated.

Chapter 9, Clustering for matplotlib, introduces parallel programming and clusters that are designed for use with matplotlib, demonstrating how to distribute the parts of a problem and then assemble the results for analysis in matplotlib.

What you need for this book

For this book, you will need Python 3.4.2 or a later version of this as is available with Ubuntu 15.04 and Mac OS X 10.10. This book was written using Python 3.4.2 on Mac OS X.

You will also need graphviz, HDF5, and their respective development libraries installed. Obtaining the code for each chapter depends upon the Git binary being present on your system. The other software packages that are used in this book will be automatically downloaded and installed for you in a virtual environment when you clone and set up the code for each chapter. Some of the chapters explore the use of matplotlib in Cloud environments. This is demonstrated by using Amazon AWS. As such, an AWS account will be needed for the users who wish to go through all the steps for these chapters.

If you are new to Python 3, the first chapter provides a brief overview of the same. It will provide you with the level of comfort that is needed when dealing with the examples in the book.

Who this book is for

If you are a scientist, programmer, software engineer, or a student who has working knowledge of matplotlib and now want to extend your usage of matplotlib to plot complex graphs and charts and handle large datasets, then this book is for you.

Conventions

In this book, you will find a number of text styles that distinguish between different kinds of information. Here are some examples of these styles and an explanation of their meaning.

Code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles are shown as follows:

"The axes and projections directories form a crucial part of the artist layer."

A block of code is set as follows:

```
#!/usr/bin/env python3.4
import matplotlib.pyplot as plt

def main () -> None:
    plt.plot([1,2,3,4])
    plt.ylabel('some numbers')
```

```
plt.savefig('simple-line.png')

if __name__ == '__main__':
    main()
```

Any command-line input or output is written as follows:

```
$ git clone https://github.com/masteringmatplotlib/architecture.git
$ cd architecture
$ make
```

New terms and **important words** are shown in bold. Words that you see on the screen, for example, in menus or dialog boxes, appear in the text like this: " For instance, when the **Zoom-to-Rectangle** button is clicked, the mode will be set to zoom rect "

 Warnings or important notes appear in a box like this.

 Tips and tricks appear like this.

Reader feedback

Feedback from our readers is always welcome. Let us know what you think about this book – what you liked or disliked. Reader feedback is important for us as it helps us develop titles that you will really get the most out of.

To send us general feedback, simply e-mail feedback@packtpub.com, and mention the book's title in the subject of your message.

If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, see our author guide at www.packtpub.com/authors.

Customer support

Now that you are the proud owner of a Packt book, we have a number of things to help you to get the most from your purchase.

Downloading the example code

Each chapter in *Mastering matplotlib* provides instructions on obtaining the example code and notebook from Github. A master list has been provided at <https://github.com/masteringmatplotlib/notebooks>. You can download the example code files from your account at <http://www.packtpub.com> for all the Packt Publishing books you have purchased. If you purchased this book elsewhere, you can visit <http://www.packtpub.com/support> and register to have the files e-mailed directly to you.

Downloading the color images of this book

We also provide you with a PDF file that has color images of the screenshots/diagrams used in this book. The color images will help you better understand the changes in the output. You can download this file from https://www.packtpub.com/sites/default/files/downloads/75420S_ColoredImages.pdf.

Errata

Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you find a mistake in one of our books—maybe a mistake in the text or the code—we would be grateful if you could report this to us. By doing so, you can save other readers from frustration and help us improve subsequent versions of this book. If you find any errata, please report them by visiting <http://www.packtpub.com/submit-errata>, selecting your book, clicking on the **Errata Submission Form** link, and entering the details of your errata. Once your errata are verified, your submission will be accepted and the errata will be uploaded to our website or added to any list of existing errata under the Errata section of that title.

To view the previously submitted errata, go to <https://www.packtpub.com/books/content/support> and enter the name of the book in the search field. The required information will appear under the **Errata** section.

Piracy

Piracy of copyrighted material on the Internet is an ongoing problem across all media. At Packt, we take the protection of our copyright and licenses very seriously. If you come across any illegal copies of our works in any form on the Internet, please provide us with the location address or website name immediately so that we can pursue a remedy.

Please contact us at copyright@packtpub.com with a link to the suspected pirated material.

We appreciate your help in protecting our authors and our ability to bring you valuable content.

Questions

If you have a problem with any aspect of this book, you can contact us at questions@packtpub.com, and we will do our best to address the problem.

1

Getting Up to Speed

Over the past 12 years of its existence, **matplotlib** has made its way into the classrooms, labs, and hearts of the scientific computing world. With Python's rise in popularity for serious professional and academic work, matplotlib has taken a respected seat beside long-standing giants such as Mathematica by Wolfram Research and MathWorks' MATLAB products. As such, we feel that the time is ripe for an advanced text on matplotlib that guides its more sophisticated users into new territory by not only allowing them to become experts in their own right, but also providing a clear path that will help them apply their new knowledge in a number of environments.

As a part of a master class series by Packt Publishing, this book focuses almost entirely on a select few of the most requested advanced topics in the world of matplotlib, which includes everything from matplotlib internals to high-performance computing environments. In order to best support this, we want to make sure that our readers have a chance to prepare for the material of this book, so we will start off gently.

The topics covered in this chapter include the following:

- A brief historical overview of matplotlib
- What's new in matplotlib
- Who is an advanced, beginner, or an intermediate matplotlib user
- The software dependencies for many of the book's examples
- An overview of Python 3
- An overview of the coding style used in this book
- References for installation-related instructions
- A refresher on IPython Notebooks
- A teaser of a complicated plot in matplotlib
- Additional resources to obtain advanced beginner and intermediate matplotlib knowledge

A brief historical overview of matplotlib

The open source project that we now know as matplotlib had its inception at the beginning of the millennium when John Hunter and his colleagues were conducting epilepsy research using proprietary data analysis software. They migrated to MATLAB as it was more flexible and less expensive. However, it was not designed to handle the data formats and diverse data sources that they had to contend with on a daily basis.

It was with this realization that John Hunter created the first version of matplotlib—a **GTK+** visualization tool for *electroencephalography* and *electrocorticography* analysis. Having been built in Python, adding support for new features as the team needed them was a straightforward task. Before long, this led to the idea of providing a similar interactive command mode to generate plots on the fly, as MATLAB does.

One of the oldest sources available for matplotlib code online is the GitHub repository. The first commit in this repository was with regard to migration from Subversion to Git, though the original repository was CVS. This commit was authored in May 2003, though this repository records a `CHANGELOG` file whose first entry was made in December 2002. By the time this book goes into publication, matplotlib will have celebrated its 13th birthday.

What's new in matplotlib 1.4

In the past 12 years, a great deal has happened in the matplotlib codebase. Of particular interest are the new features that have been added to the most recent release at the time of writing this book—version 1.4.3. Here are some of its highlights:

- A new IPython Notebook backend for interactive matplotlib plot support
- A new style package that allows for greater control over the visual presentation of plots
- The new **Qt5** backend
- Google App Engine integration
- New plotting features
- New configuration options

The intermediate matplotlib user

If you've read the preface, then you know who this book is for—developers with intermediate or advanced knowledge of matplotlib as well as the motivated beginners. But who are they exactly? What do such users know?

Answers to such questions are fairly open-ended. We have the following guidelines. The intermediate matplotlib user should have some limited knowledge to passing experience with the following:

- Installation of matplotlib in multiple environments
- Creation of basic to moderately complicated matplotlib plots
- Basic matplotlib APIs, styling, backends, and customizations
- Using matplotlib objects, subplots, and overlays
- Advanced third-party tools such as **Seaborn**, **Pandas**, **ggplot**, distributed **IPython**, and **StarCluster**
- Completed reading most or all of the following books, *Matplotlib for Python Developers*, Sandro Tosi, Packt Publishing, and *matplotlib Plotting Cookbook*, Alexandre Devert, Packt Publishing

Prerequisites for this book

This book assumes that you have previous experience with matplotlib and that it has been installed on your preferred development platform. If you need a refresher on the steps to accomplish that, the first chapter of Sandro Tosi's excellent book, *Matplotlib for Python Developers*, provides instructions to install matplotlib and its dependencies.

In addition to matplotlib, you will need a recent installation of IPython to run many of the examples and exercises provided. For help in getting started with IPython, there many great resources available on the project's site. Cyrille Rossant has authored *Learning IPython for Interactive Computing and Data Visualization*, Packt Publishing, which is a great resource as well.

In the course of this book, we will install, configure, and use additional open source libraries and frameworks. We will cover the setup of these as we get to them, but all the programs in this book will require you to have the following installed on your machine:

- **Git**
- **GNU make**
- **GNU Compiler Collection (gcc)**

Your operating system's package manager should have a package that installs common developer tools—these tools should be installed as well, and may provide most of the tools automatically.

All the examples in this book will be implemented using a recent release of Python, version 3.4.2. Many of the examples will not work with the older versions of Python, so please note this carefully. In particular, the setup of virtual environments uses a feature that is new in Python 3.4.2, and some examples use the new type annotations. At the time of writing this book, the latest version of Ubuntu ships with Python 3.4.2.

Though matplotlib, NumPy, IPython, and the other libraries will be installed for you by set scripts provided in the code repositories for each chapter. For the sake of clarity, we will mention the versions used for some of these here:

- matplotlib 1.4.3
- NumPy 1.9.2
- SciPy 0.15.1
- IPython 3.1.0 (also known as *Jupyter*)

Python 3

On this note, it's probably good to discuss Python 3 briefly as there has been continued debate on the choice between the two most recent versions of the programming language (the other being the 2.7.x series). Python 3 represents a massive community-wide effort to adopt better coding practices as well as improvements in the maintenance of long-lived libraries, frameworks, and applications. The primary impetus and on-going strength of this effort, though, is a general overhaul of the mechanisms underlying Python itself. This will ultimately allow the Python programming language greater maintainability and longevity in the coming years, not to mention better support for the ongoing performance enhancements.

In case you are new to Python 3, the following table, which compares some of the major syntactical differences between Python 2 and Python 3, has been provided:

Syntactical Differences	Python 2	Python 3
Division with floats	<code>x = 15 / 3.0</code>	<code>x = 15 / 3</code>
Division with truncation	<code>x = 15 / 4</code>	<code>x = 15 // 4</code>
Longs	<code>y = long(x * 10)</code>	<code>y = int(x * 10)</code>
Not equal	<code>x <> y</code>	<code>x != y</code>
The unicode function	<code>u = unicode(s)</code>	<code>u = str(s)</code>
Raw unicode	<code>u = ur"\t\s"</code>	<code>u = r"\t\s"</code>

Syntactical Differences	Python 2	Python 3
Printing	<code>print x, y, z</code>	<code>print(x, y, z)</code>
Raw user input	<code>y = raw_input(x)</code>	<code>y = input(x)</code>
User input	<code>y = input(x)</code>	<code>y = eval(input(x))</code>
Formatting	<code>"%d %s" % (n, s)</code>	<code>"{} {}".format(n, s)</code>
Representation	<code>'x'</code>	<code>repr(x)</code>
Function application	<code>apply(fn, args)</code>	<code>fn(*args)</code>
Filter	<code>itertools.ifilter</code>	<code>filter</code>
Map	<code>itertools.imap</code>	<code>map</code>
Zip	<code>itertools.izip</code>	<code>zip</code>
Range	<code>xrange</code>	<code>range</code>
Reduce	<code>reduce</code>	<code>functools.reduce</code>
Iteration	<code>iterator.next()</code>	<code>next(iterator)</code>
The execute code	<code>exec code</code>	<code>exec(code)</code>
The execute file	<code>execfile(file)</code>	<code>exec(fh.read())</code>
Exceptions	<pre>try: ... except val, err: ...</pre>	<pre>try: ... except val as err: ...</pre>

Coding style

The coding style used throughout this book and in the example code conforms to the standards laid out in **PEP 8**, with one exception. When entering code into an IPython Notebook or providing modules that will be displayed in the notebook, we will not use two lines to separate what would be module-level blocks of code. We will just use one line. This is done to save screen space.

Something that might strike you as different in our code is the use of an extraordinary feature of Python 3—*function annotations*. The work for this was done in **PEP 3107** and was added in the first release of Python 3. The use of types and static analysis in programming, though new to Python, is a boon to the world of software. It saves time in development of a program by catching bugs before they even arise as well as streamlining unit tests. The benefit of this in our particular case, with regard to

the examples in this book, is quick, intuitive code clarification. When you look at the functions, you will instantly know what is being passed and returned.

Finally, there is one best practice that we adhere to that is not widely adopted in the Python programming community – functions and methods are kept small in all of our code. If more than one logical thing is happening in a function, we break it into multiple functions and compose as needed. This keeps the code clean and clear, making examples much easier to read. It also makes it much easier to write unit tests without some of the excessive parameterization or awkward, large functions and methods that are often required in unit tests. We hope that this leaves a positive, long-lasting impression on you so that this practice receives wider adoption.

Installing matplotlib

Given that this is a book on an advanced topic and the target audience will have installed matplotlib and the related dependencies more than once (most likely many times), detailed instructions will not be provided here. Two excellent books on matplotlib that cover this topic in their respective first chapters are *Matplotlib for Python Developers* and *matplotlib Plotting Cookbook*.

That being said, each chapter will have its own Git repository with scripts to install dependencies and set up Python's virtual environments. These scripts are a great resource, and reading them should provide additional details to those who seek to know more about installing matplotlib and the related libraries in Python virtual environments.

Using IPython Notebooks with matplotlib

Python virtual environments are the recommended way of working with Python projects. They keep your system, Python, and default libraries safe from disruption. We will continue this tradition in this book, but you are welcome to transcend tradition and utilize the matplotlib library and the provided code in whatever way you see fit.

Using the native `venv` Python environment management package, each project may define its own versions of dependent libraries, including those of matplotlib and IPython. The sample code for this book does just that – listing the dependencies in one or more `requirements.txt` files.

With the addition of the `nbagg` IPython Notebook backend to matplotlib in version 1.4, users can now work with plots in a browser very much like they've been able to do in the GTK and Qt apps on the desktop. We will take full advantage of this new feature.

In the IPython examples of this book, most of the notebooks will start off with the following:

```
In [1]: import matplotlib
        matplotlib.use('nbagg')

In [2]: %matplotlib inline

In [3]: import matplotlib.pyplot as plt
```



Downloading the example code

Each chapter in *Mastering matplotlib* provides instructions on obtaining the example code and notebook from Github. A master list has been provided at <https://github.com/masteringmatplotlib/notebooks>. You can download the example code files from your account at <http://www.packtpub.com> for all the Packt Publishing books you have purchased. If you purchased this book elsewhere, you can visit <http://www.packtpub.com/support> and register to have the files e-mailed directly to you." This configures our notebooks to use matplotlib in the way that we need. The example in the following section starts off with just those commands.

A final note about IPython – the project has recently changed its name to Jupyter in an effort to embrace the language-agnostic growth the project and community has experienced as well as the architectural changes that will make the adding of new language backends much easier. The user experience will not change (except for the better), but you will notice a different name and logo when you open the chapter notebooks for this book.

Advanced plots – a preview

To give a taste of what's to come, let's start up a matplotlib IPython Notebook and look at an example. You will need to download the example from a GitHub repository first:

```
$ git clone https://github.com/masteringmatplotlib/preview.git
$ cd preview
```

You only need to do the following in order to bootstrap an environment with all the notebook dependencies and start up the notebook server:

```
$ make
```

This will do several things for you automatically, some of which are as follows:

- Clone a support repository holding various `include` files
- Create a Python virtual environment
- Install matplotlib and other scientific computing Python modules into this virtual environment
- Start an IPython Notebook server that runs on local host
- Open a browser window and load the `preview` notebook in it

In this browser window, you can run the code yourself by selecting each code section and hitting the *Shift* and *Enter* keys to execute it. Let's go through an example.

Setting up the interactive backend

As mentioned above, our notebooks will all start with the following, as does this `preview` notebook:

```
In [1]: import matplotlib
        matplotlib.use('nbagg')
        %matplotlib inline
In [2]: import matplotlib.pyplot as plt
        import seaborn as sns
        import numpy as np
        from scipy import stats
        import pandas as pd
```

These commands do the following:

- Set up the interactive backend for plotting
- Allow us to evaluate images in-line, as opposed doing the same in a pop-up window
- Provide the standard alias to the `matplotlib.pyplot` sub package and import other packages that we will need

Joint plots with Seaborn

Our first `preview` example will take a look at the Seaborn package, an open source third-party library for data visualization and attractive statistical graphs. Seaborn depends upon not only matplotlib, but also NumPy and SciPy (among others). These were already installed for you when you ran `make` (pulled from the `requirements.txt` file).

We'll cover Seaborn palettes in more detail later in the book, so the following command is just a sample. Let's use a predefined palette with a moderate color saturation level:

```
In [3]: sns.set_palette("BuPu_d", desat=0.6)
        sns.set_context("notebook", font_scale=2.0)
```

Next, we'll generate two sets of random data (with a random seed of our choosing), one for the x axis and the other for the y axis. We're then going to plot the overlap of these distributions in a hex plot. Here are the commands for the same:

```
In [4]: np.random.seed(42424242)
In [5]: x = stats.gamma(5).rvs(420)
        y = stats.gamma(13).rvs(420)
In [6]: with sns.axes_style("white"):
        sns.jointplot(x, y, kind="hex", size=16);
```

The generated graph is as follows:

